

Practical Session 3: OpenMP

OpenMP — Factorial Computation and Scheduling

Objective

- Learn how **load imbalance** affects parallel execution.
- Compare **static vs dynamic scheduling** using OpenMP.
- Practice parallelizing **loops with uneven workloads**.

1. Problem Description

We want to compute a function (e.g., factorial) for each element in an array:

- Create an array of size N .
- First half of the array contains **small numbers** (1–5) → easy computation.
- Second half contains **larger numbers** (15–25) → heavy computation.

Goal:

- Parallelize the loop that computes factorials.
- Observe that **static scheduling** leaves one thread idle.
- Show that **dynamic scheduling** balances the load.

2. Initialization

- Create an array `arr[N]`.
- Fill **first half** with small numbers (1–5).
- Fill **second half** with larger numbers (15–25).

Hints:

- Use `rand() % 5 + 1` for small numbers.
- Use `rand() % 11 + 15` for larger numbers.
- Allocate an output array to store factorial results (use `unsigned long long`).

3. Factorial Computation

- Implement a simple factorial function:

```
unsigned long long factorial(int n) {  
    unsigned long long f = 1;  
    for (int i = 1; i <= n; i++) {  
        f *= i;  
    }  
    return f;  
}
```

- Parallelize the **loop over the array** with OpenMP.

Hints:

- Use `#pragma omp parallel for schedule(static) first`.
- Then test `#pragma omp parallel for schedule(dynamic,1)`.
- Store results in an output array: `result[i] = factorial(arr[i]);`.

4. Lab Experiments

1. Run the program with **static scheduling**.
 - Observe which thread finishes first.
2. Run with **dynamic scheduling**.
 - Observe improved load balancing.
3. Optionally try **different chunk sizes** for dynamic scheduling (`schedule(dynamic, chunk)`).

- **Measurement**

- Measure **execution time** for each scheduling strategy using `omp_get_wtime()`.

Suggested table format:

Scheduling	Chunk Size	Thread Count	Execution Time (s)
Static	-	2	
Dynamic	1	2	
Dynamic	5	2	

6. Discussion Questions

1. Why does **static scheduling** cause one thread to finish faster?
2. How does **dynamic scheduling** improve performance?
3. What happens if all numbers in the array are small? Does static scheduling still cause imbalance?
4. How does the **chunk size** in dynamic scheduling affect execution?
5. Could you use a **reduction** if the goal was to compute the sum of factorials instead?

7. Extra Question: Optimizing Factorial Computation

Problem:

Currently, each factorial is computed from scratch in the loop:

```
result[i] = factorial(arr[i]);
```

Observation:

- Factorials are cumulative: $n! = n \times (n-1)!$
- If you already computed $(n-1)!$, you can compute $n!$ **faster**.

Task:

- Modify the program to **store all factorials found so far** in an array.
- When computing `factorial(n)`, **start from the largest previously computed factorial** instead of 1.
- Example: if $5!$ is known, then $6! = 6 \times 5!$.

Hints:

1. Keep a shared array `factorial_cache[n_max+1]` initialized to 0.
2. Before computing `factorial(n)`, check `factorial_cache[n]`.
3. Compute missing factorials **iteratively from the largest cached value**.
4. Use proper **synchronization** if multiple threads update the cache (e.g., `#pragma omp critical` or `atomic`).

Questions for Discussion:

1. How much faster is this approach compared to computing each factorial from scratch?
2. What synchronization issues arise when multiple threads try to use/update the cache?
3. How does dynamic scheduling interact with this optimization?